

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 2



Jonathan Andrew Wijaya

109082500208

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Array : Array merupakan struktur data linear yang berfungsi untuk menampung sekumpulan elemen dengan tipe data yang seragam. Dalam bahasa Go, array memiliki karakteristik statis, yang berarti ukuran atau jumlah elemen maksimalnya sudah ditetapkan secara mutlak pada saat deklarasi variabel dan tidak dapat diubah selama program dieksekusi. Setiap elemen di dalam array menempati ruang memori yang berurutan dan dapat diakses dengan sangat cepat menggunakan sistem indeks numerik, di mana perhitungan indeks selalu diawali dari angka 0. Struktur ini sangat efisien ketika kita perlu mengelola koleksi data yang batasan ukurannya sudah diketahui pasti sejak awal.

Struct : *Struct* atau tipe data bentukan adalah fitur pemrograman yang memungkinkan pengembang untuk menggabungkan beberapa variabel dengan tipe data yang berbeda-beda ke dalam satu entitas tunggal yang terstruktur. Berbeda dengan array yang mensyaratkan tipe data yang sama untuk semua elemennya, *struct* sangat ideal untuk memodelkan objek dunia nyata yang lebih kompleks. Sebagai contoh, *struct* dapat digunakan untuk mendefinisikan entitas "Mahasiswa" yang secara bersamaan menyimpan atribut teks (Nama, NIM) dan angka desimal (IPK). Penggunaan *struct* membuat kode menjadi lebih rapi, terpusat, dan mudah dikelola layaknya konsep dasar dalam pemrograman berorientasi objek.

MinMax : Algoritma MinMax adalah teknik penelusuran (traversal) dasar yang digunakan untuk mengidentifikasi nilai paling kecil (minimum) atau nilai paling besar (maksimum) dari sebuah himpunan data. Logika kerjanya cukup sederhana: algoritma akan menjadikan elemen pertama dari array (indeks ke-0) sebagai nilai minimum atau maksimum sementara. Setelah itu, program akan melakukan iterasi berurutan ke sisa elemen lainnya; setiap kali menemukan elemen yang nilainya lebih kecil dari batas minimum atau lebih besar dari batas maksimum sementara, variabel penampung tersebut akan langsung ditimpa (diperbarui) dengan nilai yang baru. Pada akhir iterasi, variabel penampung tersebut akan menyimpan nilai ekstrim yang mutlak dari keseluruhan data.

B. Guided

1. MinMax Array

```
package main

import "fmt"

func nilaiTerbesar(array []int) (int, int) {
    var idx int
    var nilaiTerbesar int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] > nilaiTerbesar {
            nilaiTerbesar = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerbesar, idxDitemukan
}

func nilaiTerkecil(array []int) (int, int) {
    var idx int
    var nilaiTerkecil int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] < nilaiTerkecil {
            nilaiTerkecil = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerkecil, idxDitemukan
}

func main() {
    var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}

    var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])
    var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])
}
```

```

    fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d",
nilaiMin, idxNilaiMin)
    fmt.Println()
    fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-%d",
nilaiMaks, idxNilaiMaks)
}

```

Output :

```

Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2

```

Penjelasan:

Program ini mendemonstrasikan algoritma sederhana untuk mencari nilai paling besar dan paling kecil di dalam sebuah deret angka beserta posisi indeksinya. Kode ini secara modular dipisah menjadi dua fungsi utama (nilaiTerbesar dan nilaiTerkecil), di mana keduanya bekerja dengan menelusuri elemen-elemen slice menggunakan perulangan dan membandingkannya satu per satu. Pada fungsi main, sebuah array statis dideklarasikan dan seluruh isinya dioper ke dalam fungsi tersebut menggunakan teknik slicing [:] yang akan menyalin seluruh elemen dari array aslinya. Pada akhirnya, program mencetak angka terbesar dan terkecil yang berhasil ditemukan beserta letak indeksinya.

2. MinMax Struct

```

package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   string
    IPK   float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0]
    var idxDitemukan int = 0

```

```

var i int

for i = 1; i < len(array); i++ {
    if array[i].IPK > mhsTerbesar.IPK {
        mhsTerbesar = array[i]
        idxDitemukan = i
    }
}

return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i, j int
    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkan NIM :")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("=====")
    }

    fmt.Println()
    for j = 0; j < len(arrMhs); j++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", j)
    }
}

```

```

        fmt.Println("Nama : ", arrMhs[j].nama)
        fmt.Println("NIM : ", arrMhs[j].nim)
        fmt.Println("IPK : ", arrMhs[j].IPK)
        fmt.Println("=====")
    }
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])
    mhsMin, idxMin = IPKTerkecil(arrMhs[:])

    fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")
    fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s,
ditemukan pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim,
idxMaks)
    fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s,
ditemukan pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim,
idxMin)
}

```

Output :

```

DATA MAHASISWA INDEKS KE- 1
Nama : Thanos
NIM : 100000789
IPK : 3.8
=====
DATA MAHASISWA INDEKS KE- 2
Nama : Stark
NIM : 10000000
IPK : 4
=====
=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 4.00, atas nama Stark dengan NIM 10000000, ditemukan pada indeks ke-2
IPK Terkecil : 2.70, atas nama Thor dengan NIM 1000082400, dit
emukan pada indeks ke-0

```

Penjelasan :

Program ini mendemonstrasikan pemanfaatan tipe bentukan (*struct*) yang disusun di dalam sebuah array statis untuk mengelola sekumpulan entitas data mahasiswa secara terstruktur. Algoritma di dalamnya meminta pengguna memasukkan data nama, NIM, dan IPK secara interaktif, untuk kemudian ditampilkan kembali sebagai rekapitulasi. Melalui fungsi modular yang menerima parameter *slice*, program menelusuri atribut IPK pada masing-masing elemen untuk mencari dan mencetak secara spesifik data mahasiswa yang meraih nilai tertinggi dan terendah beserta posisi indeksinya.

3. [judul guided 3]

[isi dengan source code]

Output :

[masukkan screenshot output]

C. Unguided

1. Soal Unguided 1

Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    var n int
    var beratKelinci [1000]float64

    fmt.Scan(&n)

    if n <= 0 {
        return
    }

    for i := 0; i < n; i++ {
        fmt.Scan(&beratKelinci[i])
    }

    min := beratKelinci[0]
    max := beratKelinci[0]

    for i := 1; i < n; i++ {
```

```

        if beratKelinci[i] < min {
            min = beratKelinci[i]
        }
        if beratKelinci[i] > max {
            max = beratKelinci[i]
        }
    }

    fmt.Printf("%.2f %.2f\n", min, max)
}

```

Output :

```

5
1.2 3.5 0.8 2.4 1.5
0.80 3.50

```

Penjelasan :

Program ini berfungsi untuk mencari data berat kelinci paling ringan (minimum) dan paling berat (maksimum) dari sekumpulan data masukan. Algoritma bekerja dengan membaca jumlah kelinci beserta berat masing-masing ke dalam sebuah *array*, lalu menjadikan data pertama sebagai patokan nilai awal. Program kemudian menelusuri sisa elemen di dalam *array* tersebut untuk memperbarui nilai minimum dan maksimum, dan mencetak hasil akhirnya dengan format dua angka di belakang koma.

2. Soal Unguided 2

Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

Jawaban :

```

package main

import (
    "fmt"
)

func main() {

```



```

var x, y int
var beratIkan [1000]float64

fmt.Scan(&x, &y)

for i := 0; i < x; i++ {
    fmt.Scan(&beratIkan[i])
}

var totalWadah []float64
var beratSementara float64
var hitungIkan int

for i := 0; i < x; i++ {
    beratSementara += beratIkan[i]
    hitungIkan++

    if hitungIkan == y || i == x-1 {
        totalWadah = append(totalWadah, beratSementara)
        beratSementara = 0
        hitungIkan = 0
    }
}

var totalSemuaWadah float64

for i := 0; i < len(totalWadah); i++ {
    fmt.Printf("%.2f", totalWadah[i])
    if i < len(totalWadah)-1 {
        fmt.Print(" ")
    }
    totalSemuaWadah += totalWadah[i]
}
fmt.Println()

rataRata := totalSemuaWadah / float64(len(totalWadah))
fmt.Printf("%.2f\n", rataRata)
}

```

Output :

```
5 2
1.5 2.0 1.0 3.0 2.5
3.50 4.00 2.50
3.33
```

Penjelasan :

Program ini berfungsi untuk menghitung total berat ikan di dalam setiap wadah beserta rata-rata berat seluruh wadah. Algoritma bekerja dengan menjumlahkan berat ikan secara berurutan dan akan memotong/menyimpan hasil jumlahnya ke dalam sebuah *slice* (`totalWadah`) setiap kali kapasitas wadah (`y`) sudah terpenuhi atau jika ikan sudah habis. Pada akhir eksekusi, program mencetak total berat dari masing-masing wadah dan membaginya dengan jumlah wadah yang terpakai untuk mendapatkan nilai rata-ratanya.

3. Soal Unguided 3

Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Jawaban :

```
package main

import (
    "fmt"
)

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax *float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
```

```

        *bMax = arrBerat[i]
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    var total float64
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var n int
    var dataBalita arrBalita

    fmt.Print("Masukan banyak data berat balita: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d: ", i+1)
        fmt.Scan(&dataBalita[i])
    }

    var min, max float64

    if n > 0 {
        hitungMinMax(dataBalita, n, &min, &max)
        rataRata := rerata(dataBalita, n)

        fmt.Printf("Berat balita minimum: %.2f kg\n", min)
        fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
        fmt.Printf("Rerata berat balita: %.2f\n", rataRata)
    }
}

```

Output :

```
PS C:\Users\COLORFUL\Documents\ALPROTELKOM> go run "c:\Users\COLORFUL\Documents\ALPROTELKOM\Guided\Unguided3.go"
Masukan banyak data berat balita: 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38
```

Penjelasan :

Program ini dirancang untuk mencatat dan menganalisis data berat badan balita. Algoritma bekerja dengan menyimpan sekumpulan input berat dari pengguna ke dalam sebuah array, kemudian memprosesnya melalui dua subprogram yang terpisah secara modular. Fungsi rerata bertugas mengakumulasi total berat untuk mencari nilai rata-rata, sedangkan prosedur hitungMinMax secara khusus memanfaatkan pointer (pass-by-reference) untuk menelusuri data dan memperbarui nilai minimum serta maksimum secara langsung di memori utama.

D. Kesimpulan

Kesimpulan dari praktikum ini adalah bahasa Go sangat bisa diandalkan dan efisien untuk mengolah komputasi data, seperti mencari nilai minimum, maksimum, serta rata-rata. Kita membuktikan bahwa metode *Linear Search* sangat efektif dan tangguh untuk menelusuri kumpulan data yang acak atau tidak teratur. Selain itu, teknik menjadikan elemen pertama array sebagai patokan nilai awal terbukti lebih aman untuk mencegah *bug* atau kesalahan logika, dibandingkan jika kita melakukan inisialisasi menggunakan angka sembarang.

Praktikum ini juga menekankan betapa pentingnya menentukan struktur data yang paling pas dengan kebutuhan. Array statis sangat ideal untuk alokasi data yang kapasitas maksimalnya sudah pasti, sedangkan *slice* memberikan keleluasaan lebih untuk menampung data yang sifatnya dinamis. Lebih dari itu, pemanfaatan *struct* menunjukkan bahwa algoritma dasar ini bisa dikembangkan untuk menangani objek data dunia nyata

yang jauh lebih kompleks—seperti pendataan akademik mahasiswa—bukan sekadar deretan angka biasa.

Terakhir, praktik memecah program menjadi beberapa fungsi kecil (modular) terbukti sangat membantu saat kita perlu melacak *error* (*debugging*) dan melakukan perawatan kode. Kita juga berhasil mengimplementasikan *pointer* untuk memanipulasi memori tingkat lanjut, di mana sebuah subprogram dapat langsung mengubah nilai asli dari variabel di memori utama. Pada intinya, penguasaan komprehensif terhadap perulangan, percabangan, struktur data, dan pengelolaan memori adalah fondasi wajib untuk menciptakan program yang efisien dan mudah dikembangkan (*scalable*).